

https://www.kaggle.com/datasets/xdxd003/ff-c23?utm_source=chatgpt.com (18gb)

https://www.kaggle.com/datasets/reubensuju/celeb-df-v2?utm_source=chatgpt.com (10gb)

https://www.kaggle.com/datasets/selfishgene/synthetic-faces-high-quality-sfhq-part-1?utm_source=chatgpt.com (15 gb)

Kaggle API (kaggle) – Direct dataset downloads from Kaggle.

PyTorch + Hugging Face (model training & pretrained models)

OpenCV + Albumentations (image preprocessing & augmentation)

Piexif + ExifRead (metadata analysis)

scikit-learn + Grad-CAM (evaluation + explainability)

Flask/Streamlit/Gradio (deployment as web app)

◆ 3. Collaboration Workflow

1. **Branching model** → each team member works on a separate branch (`data-prep`, `model-training`, `web-app`).
2. **Pull requests** → review before merging into `main`.
3. **Issues/Projects** → assign tasks (datasets, training, UI, etc.).

◆ 5. Deployment Plan

- After training in Kaggle/Colab, download the **trained model weights (.pth/.pt)**.
- Push them to **GitHub Releases** (if small) or **Google Drive/Kaggle dataset** (if large).
- Build your **Streamlit/Gradio app locally** → everyone in your group can run it.

php

 Copy code

```
deepfake-detection/
├── data/                                # (empty) -> datasets handled via Kaggle, not uploaded
├── notebooks/                          # Jupyter/Colab notebooks for experiments
│   ├── 01_data_exploration.ipynb
│   ├── 02_model_training.ipynb
│   └── 03_metadata_analysis.ipynb
├── src/                                # Source code for project
│   ├── __init__.py
│   ├── data_preprocessing.py           # preprocessing, augmentation
│   ├── metadata_utils.py               # EXIF, steganography analysis
│   ├── model.py                       # PyTorch/HF model architecture
│   ├── train.py                       # training loop
│   ├── evaluate.py                    # metrics + Grad-CAM visualization
│   └── inference.py                   # load model & predict real/fake
├── webapp/                             # Deployment
│   ├── app.py                         # Flask / Streamlit / Gradio app
│   ├── templates/                     # (for Flask if needed)
│   └── static/                         # CSS/JS files
├── tests/                              # Unit tests
│   ├── test_model.py
│   ├── test_metadata.py
│   └── test_preprocessing.py
├── .gitignore                          # Ignore large files, datasets, venv
├── requirements.txt                    # Python dependencies
├── environment.yml                     # (optional, for conda users)
├── kaggle.json                         # (ignored, add to .gitignore)
├── README.md                           # Project description & setup
└── LICENSE                             # Open-source license (MIT recommended)
```

4. requirements.txt (sample)

```
torch
torchvision
torchaudio
transformers
diffusers
```

scikit-learn
opencv-python
albumentations
pillow
piexif
ExifRead
stegano
matplotlib
seaborn
tqdm
flask
streamlit
gradio

◆ Project Phases & Task Breakdown

📌 Phase 1: Setup & Planning

- Create GitHub repository with proper structure (`main` , `dev` , feature branches).
- Add `.gitignore` , `requirements.txt` , and `README.md` .
- Set up **Kaggle API** in repo (don't commit `kaggle.json`).
- Create GitHub Project board (columns: *To-Do*, *In Progress*, *Review*, *Done*).

👤 **Assign:** *All team members* (initial setup).

📌 Phase 2: Data Handling

- Explore available Kaggle datasets (FaceForensics++, Celeb-DF, DFDC, Synthetic faces).
- Write a script (`data_preprocessing.py`) for:
 - Downloading datasets from Kaggle.
 - Cleaning & organizing data into `train/val/test` .
 - Applying augmentations (Albumentations).
- Small subset selection (to avoid huge downloads).

👤 **Assign:** *Rohan* (since you already know Kaggle API well).

📌 Phase 3: Model Development

- Implement baseline CNN (ResNet18 / EfficientNet).
- Integrate Hugging Face Vision Transformer (ViT / CLIP).
- Add training loop (`train.py`) with PyTorch Lightning or custom trainer.
- Save trained model weights (`.pth`).

👤 **Assign:** *Kamal* (deep learning focus).

Phase 4: Metadata Analysis

-  Write utilities (`metadata_utils.py`) to extract EXIF tags with **Piexif / ExifRead**.
-  Create functions to detect inconsistencies (e.g., missing camera info, edited timestamps).
-  (Optional) Integrate steganography detection using **Stegano**.
-  Combine metadata features with CNN output (late fusion).

 **Assign:** Govardhan (metadata & utility coding).

Phase 5: Evaluation & Explainability

-  Use **scikit-learn metrics** → accuracy, precision, recall, F1, ROC.
-  Implement **Grad-CAM** visualization (`evaluate.py`).
-  Compare results: Pixel-only vs. Pixel+Metadata models.
-  Store evaluation plots in `/notebooks/03_metadata_analysis.ipynb`.

 **Assign:** Rohan + Kamal (joint, since one has ML, one has DL).

Phase 6: Deployment

-  Create **Streamlit/Gradio app** (`webapp/app.py`):
 - Upload image → Run detection → Show confidence score.
 - Display Grad-CAM heatmap.
 - Show metadata inconsistencies.
-  (Optional) Build **Flask API** for scalability.
-  (Optional) Plan for **browser extension prototype**.

 **Assign:** Govardhan + Rohan (UI + API).

Phase 7: Documentation & Final Touch

-  Update `README.md` with usage instructions.
-  Add comments & docstrings in all code files.
-  Write project **report/summary** (can use repo Wiki).
-  Prepare presentation/demo (with live Streamlit/Gradio app).

 **Assign:** All team members.

Suggested GitHub Project Board Columns

1. **To-Do** → all tasks above.
 2. **In Progress** → current work.
 3. **Review** → code in PRs awaiting review.
 4. **Done** → merged into `dev` or `main`.
-

Example Assignment Snapshot

Phase	Task Owner(s)	
Setup & Planning	All	
Data Handling	Rohan	
Model Development	Kamal	
Metadata Analysis	Govardhan	
Evaluation/Explain	Rohan + Kamal	
Deployment	Govardhan + Rohan	

